



Cross-Origin Resource Sharing

I. CORS và Cơ Chế Same-Origin Policy

CORS (Cross-Origin Resource Sharing) là một cơ chế cho phép các trang web truy cập tài nguyên từ miền khác với miền của nó (cross-origin) một cách an toàn. Trước khi có CORS, Same-Origin Policy (SOP) là quy tắc bảo mật của trình duyệt để ngăn tài nguyên web từ miền này truy cập miền khác nhằm bảo vệ người dùng khỏi các tấn công như Cross-Site Scripting (XSS).

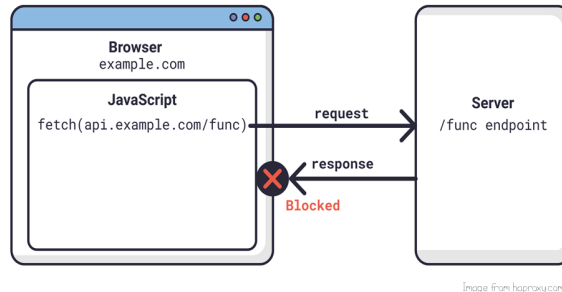
Same-Origin Policy xác định hai trang web có cùng origin nếu cùng:

- **Protocol** (HTTP, HTTPS),
- **Host** (tên miền),
- **Port** (80, 443, v.v.).

SOP hạn chế các request giữa các miền khác nhau, nhưng không hoàn toàn chặn các cuộc gọi giữa miền khác nếu không đòi hỏi quyền truy cập dữ liệu, như khi sử dụng thẻ hình ảnh hoặc script. Tuy nhiên, nó ngăn chặn trang web đọc dữ liệu phản hồi từ miền khác để bảo vệ thông tin người dùng.

II. CORS Hoạt Động Như Thế Nào?

CORS Explained for Beginners



CORS được thiết kế để hỗ trợ các tương tác cross-origin khi các miền tin cậy lẫn nhau. Khi trang web gửi một yêu cầu cross-origin, trình duyệt sẽ tự động thêm Origin header chứa nguồn gốc của yêu cầu. Dưới đây là cách các thành phần của CORS hoạt động:

1. Request từ Client:

- Client gửi request chứa Origin header tới server, ví dụ: Origin: `https://client.com`.

2. Kiểm tra của Server:

- Server kiểm tra Origin và nếu hợp lệ, nó phản hồi với `Access-Control-Allow-Origin` và cho phép client nhận dữ liệu.
- `Access-Control-Allow-Origin` thường được gán một miền cụ thể hoặc `*` (cho phép mọi nguồn), nhưng sử dụng `*` có thể không an toàn nếu dữ liệu nhạy cảm.

3. Header Quan Trọng của CORS:

- **Access-Control-Allow-Origin:** Chỉ định miền nào được phép truy xuất tài nguyên.
- **Access-Control-Allow-Methods:** Danh sách các phương thức HTTP được phép, như GET, POST, DELETE.
- **Access-Control-Allow-Headers:** Các request headers cụ thể mà server hỗ trợ.



- **Access-Control-Allow-Credentials:** Khi đặt true, cho phép gửi cookies và xác thực cross-origin.
- **Access-Control-Expose-Headers:** Liệt kê các headers mà client có thể đọc từ phản hồi cross-origin.

III. Preflight Request

Preflight request là yêu cầu kiểm tra gửi trước từ trình duyệt tới server để xác minh xem yêu cầu cross-origin có hợp lệ không. Điều này xảy ra khi yêu cầu bao gồm:

- Phương thức không phải là GET, POST, hoặc HEAD,
- Yêu cầu đính kèm headers tùy chỉnh.

Ví dụ, client gửi OPTIONS request tới server:

```
OPTIONS /resource/foo HTTP/1.1
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: origin, x-requested-with
Origin: https://foo.client.com
```

Server phản hồi với các thông tin về quyền truy cập:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://foo.client.com
Access-Control-Allow-Methods: GET, POST, DELETE
Access-Control-Max-Age: 86400
```

IV. Các Lỗi và Rủi Ro CORS Thường Gặp

Sai cấu hình CORS có thể dẫn đến nhiều rủi ro bảo mật:

1. Reflection Header Mọi Origin:



- Một số server đọc Origin từ request và phản hồi Access-Control-Allow-Origin bằng chính giá trị đó mà không kiểm tra. Điều này cho phép bất kỳ trang web nào truy cập dữ liệu nếu có header Access-Control-Allow-Credentials: true.

2. Dùng Wildcard * Cho Tất Cả Origins:

- Khi Access-Control-Allow-Origin: * được sử dụng, server mở quyền truy cập cho tất cả các trang web mà không phân biệt, dẫn đến nguy cơ lộ thông tin nhạy cảm khi các tài nguyên không được bảo vệ.

3. Whitelisting Sai Các Subdomain:

- Ví dụ, cho phép tất cả các miền có đuôi .mydomain.com có thể dẫn đến rủi ro nếu kẻ tấn công đăng ký tên miền hackermydomain.com.

4. Cho Phép Origin null:

- Origin null có thể xuất hiện khi yêu cầu cross-origin từ sandbox hoặc file nội bộ. Điều này có thể bị kẻ tấn công lợi dụng để truy cập dữ liệu nhạy cảm từ các sandbox hay tài liệu nội bộ.

V. Các Biện Pháp Bảo Mật để Tránh Lỗi CORS

Để ngăn ngừa các rủi ro bảo mật do CORS, nên cấu hình chính xác:

1. Hạn Chế Miền Được Phép:

- Chỉ cho phép các miền tin cậy trong Access-Control-Allow-Origin. Tránh phản hồi tất cả các Origin hoặc sử dụng * cho dữ liệu nhạy cảm.

2. Tránh Origin null:

- Không nên sử dụng null làm giá trị cho Access-Control-Allow-Origin, trừ khi server được bảo mật tốt và không chứa dữ liệu nhạy cảm.

3. Tránh Wildcards trên Mạng Nội Bộ:

- Không sử dụng * trong môi trường nội bộ, vì dữ liệu nhạy cảm có thể bị truy xuất từ bên ngoài qua các trình duyệt trong mạng nội bộ.

4. Sử dụng Preflight Kiểm Tra Nghiêm Ngặt:

- Yêu cầu preflight với các request chứa phương thức hoặc headers tùy chỉnh giúp kiểm soát chặt chẽ hơn các tương tác cross-origin.



5. Không Dùng CORS Thay Thế Cho Bảo Mật Máy Chủ:

- CORS chỉ là cơ chế bảo vệ phía trình duyệt; server vẫn nên áp dụng các biện pháp xác thực, kiểm tra phiên làm việc và bảo vệ dữ liệu.

VI. Cách Khắc Phục Lỗi CORS

Nếu phát sinh lỗi CORS khi phát triển ứng dụng, có thể xử lý theo một số cách:

1. Sửa Cấu Hình Server:

- Thêm domain vào Access-Control-Allow-Origin trên server để cho phép yêu cầu cross-origin hợp lệ.

2. Vô Hiệu Hóa CORS (Chỉ Dùng Khi Thử Nghiệm):

- Tạm thời vô hiệu hóa bảo mật của trình duyệt bằng cách dùng các flags như --disable-web-security trên Chrome. Tuy nhiên, cách này không an toàn và chỉ nên sử dụng trong môi trường phát triển.

3. Sử Dụng Proxy:

- Dùng server proxy đứng giữa frontend và server đích để chuyển tiếp request và response mà không gặp vấn đề CORS.

Tóm lại, CORS là công cụ quan trọng trong phát triển web hiện đại, nhưng cần cấu hình cẩn thận để bảo vệ dữ liệu người dùng và ngăn ngừa rủi ro bảo mật.

VII. Tham khảo

<https://portswigger.net/web-security/cors>

<https://viblo.asia/p/tim-hieu-ve-cross-origin-resource-sharing-cors-Az45bGWqKxY>

Mọi thắc mắc hay ý kiến đóng góp vui lòng liên hệ :

Email : r3dctf.info@gmail.com

@Design by R3D CTF Team